

No-GPS Drone Project

Technical Report

GPS-Denied Autonomous UAV Localisation & Object Detection

Location: Chiayi, Taiwan 23.4509°N, 120.2861°E

Stack: Isaac Sim 6.0 · AnyLoc · YOLOv8 · ArduPilot MAVLink

Competition: 2nd UAV Defense Challenge (ROC MND) — GPS-denied multi-target detection

Frank Lin

May 2026

Contents

1	Project Overview	2
2	System Pipeline	2
3	Module Descriptions	4
3.1	Simulator (<code>simulator/</code>)	4
3.2	Localisation — AnyLoc + Visual Odometry (<code>anyloc/</code>)	5
3.3	Object Detection — YOLOv8 (<code>detection/</code>)	6
3.4	Flight Control — ArduPilot + MAVLink (<code>control/</code>)	7
4	Repository Layout	9
5	Running the System	10
6	Milestones	11
7	Key Design Decisions	11
8	Competition Readiness	12

Project Overview

This project builds a drone system that can **localise itself and detect objects without GPS**, using visual place recognition (AnyLoc), object detection (YOLOv8), and ArduPilot for flight control. The full pipeline is validated in NVIDIA Isaac Sim before deployment to real hardware.

Motivation — Competition Context

The system is developed for the *2nd UAV Defense Application Challenge* (2nd National Defense UAV Challenge, ROC) organised by the Republic of China Ministry of National Defense. The competition theme is *GPS-denied autonomous UAV multi-target reconnaissance*: a GNSS jammer is physically mounted on the drone, and teams must still autonomously navigate, identify, and geo-locate targets.

Table 1: Competition requirements vs. project status

Requirement	Project approach	Status
GPS-denied navigation	AnyLoc + Visual Odometry	Done
Autonomous flight	ArduPilot EKF3 + GUIDED mode	In progress
Vehicle detection (cars)	YOLOv8l (VisDrone)	Done
Target geo-location	AnyLoc anchor + nadir projection	Done
Localisation accuracy (≤ 50 m)	15–20 m anchor error	Meets requirement

Software Stack

Isaac Sim NVIDIA Isaac Sim 6.0.0 (Kit 106, Python 3.12) — physics simulation, Cesium terrain, nadir camera output.

AnyLoc Universal visual place recognition — DINOv2 ViT-B/14 + VLAD + FAISS — provides GPS-free localisation at 15–20 m accuracy.

YOLOv8 `yolo8l_visdrone.pt` — YOLOv8-Large pre-trained on VisDrone 2019 DET (10 aerial vehicle classes).

ArduPilot Open-source autopilot — SITL JSON backend connects to Isaac Sim; EKF3 fuses AnyLoc position via `VISION_POSITION_ESTIMATE`.

pymavlink MAVLink protocol library — connects Python control code directly to ArduPilot over TCP.

System Pipeline

Figure 1 shows the full data flow from simulation to flight commands.

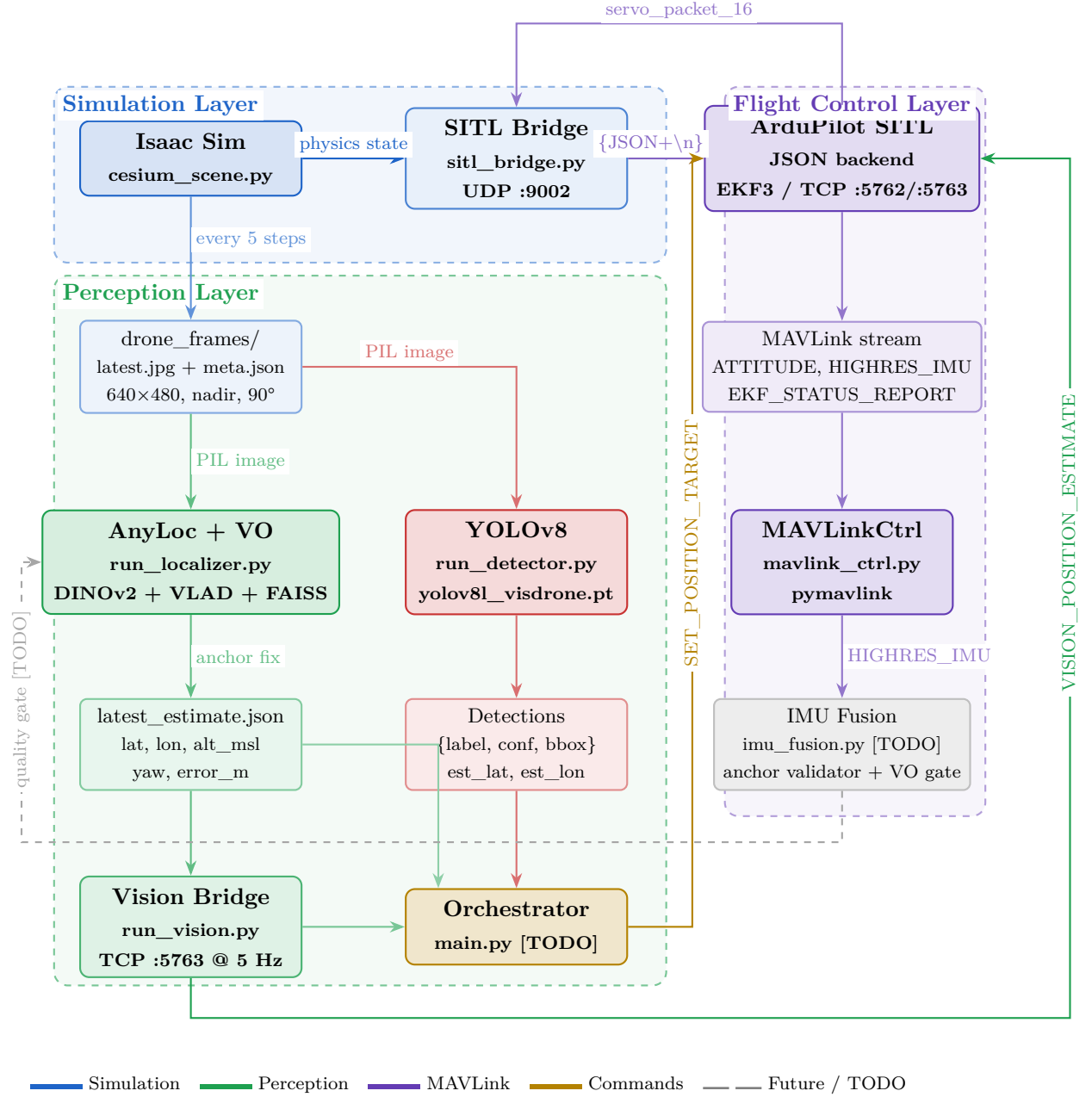


Figure 1: Full system pipeline. Three layers are colour-coded: blue (simulation loop), green (perception — localisation and detection), and purple (flight control — MAVLink and EKF3). Long feedback arrows are routed along the right margin to avoid overlaps. Dashed elements are planned future work.

Concise Data-Flow Summary

1. **Isaac Sim** renders the Chiayi, Taiwan scene and publishes nadir camera frames (`latest.jpg`) and metadata (`latest_meta.json`) every 5 simulation steps.
2. **SITL Bridge** (`sitl_bridge.py`) receives binary servo packets from ArduPilot on UDP port 9002 and replies with an IMU/velocity physics JSON packet, closing the ArduPilot simulation loop.
3. **AnyLoc** (`run_localizer.py`) runs DINOv2 + VLAD on each new camera frame, matches against a 2,821-entry database (50 m grid), and writes a geo-estimate to `anyloc/latest_estimate.json`. Visual Odometry (Lucas–Kanade) fills in position between full AnyLoc retrievals.
4. **YOLOv8** (`run_detector.py`) detects aerial vehicles in the same frame and returns bounding boxes with class labels.
5. **Vision Bridge** (`run_vision.py`) converts the AnyLoc lat/lon estimate to NED coordinates relative to the SITL home and sends `VISION_POSITION_ESTIMATE` MAVLink messages to ArduPilot EKF3 at 5 Hz.
6. **ArduPilot EKF3** fuses the vision position (configured as *ExternalNav* source, `EK3_SRC1_POSXY=6`) and reports `POS_ABS` in its EKF status flags when the fix is valid.
7. **Orchestrator** (`main.py`, `TODO`) sends `SET_POSITION_TARGET_LOCAL_NED` commands once EKF3 has a valid position, replacing manual keyboard control.

Module Descriptions

Simulator (`simulator/`)

Status: Working — Cesium terrain, NLSC orthophoto, OSM buildings, quadcopter drone, nadir camera, SITL physics bridge, HUD.

The Isaac Sim scene is centred on **Chiayi, Taiwan** at 23.4509°N, 120.2861°E.

Scene Components

Table 2: Scene data sources

Layer	Source	License
Terrain	Cesium World Terrain (asset 1), quantized-mesh-1.0	©Cesium ion
Buildings	Cesium OSM Buildings (asset 96188), B3DM	©OSM (ODbL)
Imagery	Taiwan NLSC PHOTO2 orthophoto WMTS, zoom 18	©National Land Surveying and Mapping

Drone Model

A quadcopter spanning ≈ 0.8 m is built from USD primitives under `/World/Drone`: *body* (flat cube), *four arms* at 45° intervals, *motor cylinders* at arm tips, *propeller discs* above each motor, and an *orange beacon SphereLight* (5000 cd) for visibility from the overview camera.

Camera and Frame Output

The nadir camera (`/World/Drone/Camera`) uses 18 mm focal length on a 36×27 mm aperture sensor, giving a **$90^\circ \times 73.7^\circ$ FOV**. At 50 m AGL this covers $\approx 100 \times 75$ m of ground.

Every 5 simulation steps the renderer writes:

- `drone_frames/latest.jpg` — 640×480 RGB nadir view (ML input)
- `drone_frames/latest_meta.json` — `{step, lat, lon, alt_m, agl_m, centre_elev, yaw_deg, frame_w, frame_h}`

Keyboard Controls

Key	Action
Tab	Toggle viewport: overview $\leftrightarrow$ drone nadir
W / S	Fly north / south (5 m/step)
A / D	Fly west / east
Q / E	Descend / ascend
Z / X	Yaw left / right (1° /step)

Localisation — AnyLoc + Visual Odometry (`anyloc/`)

Status: Working — 2,821-entry database (50 m grid); ≈ 15 –20 m anchor error; geo-constrained search; VO refinement to ≈ 5 –10 m between anchors.

How AnyLoc Works

AnyLoc is a *universal visual place recognition* framework. Given a query image, it extracts patch features with a pre-trained DINOv2 ViT-B/14 backbone, aggregates them into an **intra-normalised VLAD** descriptor ($k = 64$ codewords, $64 \times 768 = 49,152$ dimensions), and retrieves the nearest database entry by cosine similarity.

Database Construction

1. Place a 50 m grid (± 1500 m from scene centre) $\rightarrow$ 2,821 positions.
2. For each position, crop the NLSC satellite orthophoto at 50 m AGL (same camera FOV).
3. Extract DINOv2 features $\rightarrow$ compute VLAD $\rightarrow$ L2-normalise.
4. Store all 2,821 VLAD vectors in a FAISS `IndexFlatIP` (inner-product = cosine similarity after normalisation).

Geo-Constrained Retrieval

After the first anchor is established, each subsequent AnyLoc retrieval is **constrained to a 200 m geographic window** centred on the current VO-estimated position. Only ≈ 50 of the 2,821 database entries are compared, preventing jumps to visually similar but geographically distant tiles.

$$d_i = \sqrt{(\Delta \text{lat}_i \cdot 111,320)^2 + (\Delta \text{lon}_i \cdot 111,320 \cdot \cos \phi)^2} \quad \text{filter: } d_i \leq 200 \text{ m}$$

Visual Odometry (VO) Between Anchors

Between AnyLoc retrievals (`ANYLOC_INTERVAL` = 10 frames), `VORefiner` tracks Shi-Tomasi corner features using Lucas-Kanade optical flow. Median pixel displacement is converted to $\Delta\text{lat}/\Delta\text{lon}$ via:

$$v_{\text{raw,east}} = -\Delta x_{\text{px}} \cdot m_{\text{px}}^{-1}, \quad v_{\text{raw,north}} = +\Delta y_{\text{px}} \cdot m_{\text{px}}^{-1}$$

$$\begin{pmatrix} \Delta\text{east} \\ \Delta\text{north} \end{pmatrix} = \begin{pmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{pmatrix} \begin{pmatrix} v_{\text{raw,east}} \\ v_{\text{raw,north}} \end{pmatrix}$$

where ψ is the drone yaw and m_{px}^{-1} is metres per pixel derived from AGL and FOV.

Accuracy Table

Table 3: Localisation accuracy vs. database grid step

Grid step	Entries	Expected anchor error	Between anchors (VO)
200 m	172	≈ 65 m	—
100 m	≈ 688	≈ 30 – 40 m	—
50 m (current)	2,821	≈ 15–20 m	≈ 5–10 m
25 m	$\approx 11,000$	≈ 8 – 12 m	—

Hardware used: RTX 2080 Ti; AnyLoc inference: ≈ 183 ms per anchor frame.

Object Detection — YOLOv8 (detection/)

Status: Working — `yolov8l_visdrone.pt` active; auto class-map; fine-tuning pipeline ready.

Active Model

`yolov8l_visdrone.pt` — YOLOv8-Large pre-trained on VisDrone 2019 DET (10 aerial vehicle classes). Confidence threshold: 0.30.

Class Mapping

The `YOLODetector` class builds a class-ID-to-label mapping at load time from `model.names`, making the same code work with both COCO and VisDrone models.

Table 4: VisDrone $\rightarrow$ canonical class mapping

VisDrone class	ID	Canonical label	Box colour
car	3	car	 red
van	4	car	 red
truck	5	truck	 yellow
tricycle	6	motorcycle	 orange
awning-tricycle	7	motorcycle	 orange
bus	8	bus	 purple
motor	9	motorcycle	 orange

Fine-Tuning Pipeline

A complete pipeline adapts YOLOv8 to top-down aerial imagery:

1. `collect_training_data.py` — fly an Isaac Sim grid at 30/60/100 m AGL; export JPEG frames and YOLO labels for 43 synthetic vehicles.
2. `prepare_dataset.py` — download VisDrone 2019 DET ($\approx 7k$ images); remap to 4 classes; merge with synthetic data.
3. `finetune.py` — 100 epochs with nadir augmentations: `degrees=45`, `scale=0.5`, `mosaic=1.0`, `flipud=0.5`.

Best weights saved to `detection/runs/topdown_v1/weights/best.pt`.

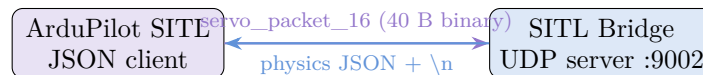
Flight Control — ArduPilot + MAVLink (control/)

Milestones 6a–6b-iii: Done — SITL JSON bridge, GPS-denied EKF3, AnyLoc vision position fully fused.

Milestones 6b-iv, 6c, 6d: TODO — autonomous flight commands, IMU reader, IMU anchor validation.

SITL JSON Bridge (`sitl_bridge.py`)

ArduPilot SITL uses a JSON physics backend where ArduPilot is the *client* and the bridge is the *server*:



The physics JSON sent each step:

```

1 {
2   "timestamp": 1234.5,
3   "imu": {
4     "gyro":      [0.0, 0.0, yaw_rate],
5     "accel_body": [sf_bx, sf_by, sf_bz]    # specific force (m/s^2)
6   },
7   "attitude":   [0.0, 0.0, yaw_rad],
8   "velocity":   [vn, ve, vd],              # required=true in SIM_JSON.h
9   "rng_1":      agl_m,
10  "battery":     {"voltage": 12.6, "current": 5.0}
11  # "position" intentionally ABSENT -- AnyLoc provides it via MAVLink
12  # instead
13 }
```

Listing 1: Physics state dict sent to ArduPilot

Key design point. The "position" field is a GPS substitute in ArduPilot's EKF3. It is deliberately omitted to enforce GPS-denied operation. "velocity" (`required=true`) stays to prevent ArduPilot from rejecting the packet.

EKF3 Configuration (no_gps.parm)

```

1 GPS_TYPE           0    # disable GPS sensor
2 EK3_SRC1_POSXY     6    # horizontal position: ExternalNav (AnyLoc)
3 EK3_SRC1_VELXY     0    # no horizontal velocity source
4 EK3_SRC1_POSZ      1    # altitude from barometer
5 EK3_SRC1_YAW       1    # yaw from compass
6 VISO_TYPE          1    # enable MAVLink ExternalNav processing

```

Listing 2: SITL parameter file for GPS-denied operation

MAVLink Controller (mavlink_ctrl.py)

MAVLinkCtrl is a non-blocking pymavlink wrapper that:

- maintains the latest HEARTBEAT, HIGHRES_IMU, ATTITUDE, LOCAL_POSITION_NED, and EKF_STATUS_REPORT in memory;
- sends VISION_POSITION_ESTIMATE (AnyLoc position $\rightarrow$ EKF3);
- exposes `arm()`, `takeoff()`, `set_position_ned()` for 6b-iv.

EKF Status Flags

Table 5: EKF_STATUS_REPORT flag bits (ArduPilot EKF3)

Constant	Bit	Hex	Meaning
EKF_ATTITUDE	0	0x0001	Attitude valid
EKF_VEL_HORIZ	1	0x0002	Horizontal velocity valid
EKF_POS_HORIZ_REL	3	0x0008	Relative horizontal position valid
EKF_POS_HORIZ_ABS	4	0x0010	Absolute position valid (vision fused) $\leftarrow target$
EKF_PRED_POS_HORIZ_ABS	9	0x0200	Vision estimate accepted
EKF_UNINITIALIZED	10	0x0400	EKF not yet initialised (normal at startup)

Vision Bridge (run_vision.py)

Reads anyloc/latest_estimate.json and converts to NED relative to SITL home:

$$N = (\phi_{\text{est}} - \phi_{\text{home}}) \cdot 111,320, \quad E = (\lambda_{\text{est}} - \lambda_{\text{home}}) \cdot 111,320 \cdot \cos \phi_{\text{home}}, \quad D = -(\text{alt} - \text{alt}_{\text{home}})$$

Sends VISION_POSITION_ESTIMATE at 5 Hz on TCP port 5763 (separate from the monitor on 5762, since each TCP port accepts one client). Covariance: 20 m horizontal std ($\approx$ AnyLoc anchor error), 5 m vertical, 0.3 rad yaw.

Confirmed result: EKF flags reach ATT,VEL_H,VEL_V,POS_REL,POS_ABS,ALT,PRED_ABS — full fusion achieved (Milestone 6b-iii).

Stub Bridge (stub_bridge.py)

A kinematic hover simulator driven by ArduPilot’s PWM outputs. Allows MAVLink testing without Isaac Sim running:

- Integrates thrust from mean motor PWM into vertical velocity and altitude.

- Horizontal position fixed at scene origin (drone hovers in place).
- Runs at 100 Hz; sends the same physics JSON as the real bridge.

Repository Layout

```

1 no_GPS_drone_project/
2 +-- instructions/
3 |   +-- project_plan.md           # module status, design decisions,
   milestones
4 |   +-- history.md                # session-by-session change log
5 +-- simulator/
6 |   +-- cesium_scene.py           # Isaac Sim scene: terrain + drone +
   camera + bridge
7 |   +-- drone_frames/             # live output: latest.jpg + latest_meta.
   json
8 |   +-- run_chiayi.sh              # launch script
9 +-- anyloc/
10 |  +-- build_database.py           # build VLAD database from NLSC orthophoto
   (once)
11 |  +-- localizer.py               # AnyLocLocalizer (DINOv2 + VLAD + FAISS)
12 |  +-- vo_refiner.py              # VORefiner (LK optical flow)
13 |  +-- run_localizer.py           # live dual postview + writes
   latest_estimate.json
14 |  +-- database/                  # 2821-entry VLAD database (49152-dim, 50
   m grid)
15 +-- detection/
16 |  +-- detector.py                 # YOLODetector (auto class-map)
17 |  +-- run_detector.py            # live annotated postview
18 |  +-- label_writer.py            # nadir projection for synthetic labels
19 |  +-- collect_training_data.py   # Isaac Sim synthetic data collector
20 |  +-- prepare_dataset.py         # VisDrone + remap + merge synth
21 |  +-- finetune.py                # YOLOv8 top-down fine-tuning
22 +-- yolov8l_visdrone.pt           # active model (VisDrone 10-class)
23 +-- control/
24 |  +-- sitl_bridge.py              # UDP :9002 server --- binary servo in,
   JSON physics out
25 |  +-- stub_bridge.py             # kinematic hover stub for MAVLink testing
26 |  +-- mavlink_ctrl.py            # MAVLinkCtrl + EKF constants + vision +
   cmds
27 |  +-- run_mavlink.py              # live terminal EKF monitor (port 5762)
28 |  +-- run_vision.py              # AnyLoc -> VISION_POSITION_ESTIMATE (port
   5763)
29 |  +-- no_gps.parm                 # SITL: GPS_TYPE=0, EK3_SRC1_POSXY=6,
   VISO_TYPE=1
30 |  +-- imu_reader.py              # HIGHRES_IMU reader [TODO 6c]
31 |  +-- imu_fusion.py              # anchor validator + VO gate [TODO 6d]
32 +-- third_party/
33 |  +-- ardupilot/                  # ArduPilot source; binary: build/sitl/bin
   /arducopter
34 +-- main.py                       # top-level orchestrator [TODO]

```

Listing 3: Project directory tree

Running the System

Requirements

- NVIDIA GPU (RTX 2080 Ti or better, 8 GB+ VRAM)
- NVIDIA driver 520+; Ubuntu 22.04 / 24.04
- conda env isaac_sim_test (Python 3.12, Isaac Sim 6.0.0)
- Cesium ion account (token embedded in `cesium_scene.py`)
- X11 display (`DISPLAY=:2` or virtual framebuffer)
- System Python packages: `pexpect mavproxy pymavlink future`

Build ArduPilot SITL (one-time)

```
cd third_party/ardupilot
git submodule update --init --depth=1 --recursive    # ~5 min
python3 waf configure --board sitl
python3 waf copter                                     # ~2 min
cd ../../
```

Launch Order (5 terminals)

```
python3 third_party/ardupilot/Tools/autotest/sim_vehicle.py \
-v ArduCopter --model=JSON --no-rebuild --console --map \
-l 23.450868,120.286135,46,0 \
--add-param-file=control/no_gps.parm \
--out tcp:localhost:5763
```

Listing 4: Terminal 1 — ArduPilot SITL

```
# Full simulation:
cd simulator && ./run_chiayi.sh

# Or lightweight stub (no Isaac Sim needed):
python3 control/stub_bridge.py
```

Listing 5: Terminal 2 — Isaac Sim (or stub)

```
DISPLAY=:2 conda run -n isaac_sim_test python anyloc/run_localizer.py
```

Listing 6: Terminal 3 — AnyLoc localizer

```
python3 control/run_vision.py
```

Listing 7: Terminal 4 — Vision bridge (AnyLoc -> EKF3)

```
python3 control/run_mavlink.py
```

Listing 8: Terminal 5 — MAVLink monitor

Expected EKF Progression

TIME flags	ROLLdeg	PCHdeg	YAWdeg	N m	E m	D m	EKF
5.0	0.00	0.00	0.00	0.00	-5.00	0x0400	UNINIT
10.0	0.01	-0.02	0.00	0.00	-5.00	0x0001	ATT
40.0	0.01	-0.02	90.00	0.12	-9.87	0x003f	ATT,VEL, POS_ABS

POS_ABS (bit 4) confirms that EKF3 has accepted the AnyLoc vision position. Flight commands will now work.

Rebuild the AnyLoc Database

Only needed after the scene or camera FOV changes:

```
conda run -n isaac_sim_test python anyloc/build_database.py --rebuild
```

Milestones

#	Milestone	Status
1	Isaac Sim scene: Cesium terrain + NLSC imagery + OSM buildings	Done
2	Virtual drone + nadir camera publishing frames	Done
3	AnyLoc map database built from simulated views	Done
4	AnyLoc localisation + dual postview on simulated frames	Done
5	YOLO detection working on simulated frames	Done
5a	Switch to VisDrone-trained YOLOv8l; auto class-map in detector	Done
5b	Top-down fine-tuning pipeline (VisDrone + synthetic data)	Ready to run
6a	ArduPilot SITL + Isaac Sim JSON bridge (IMU + baro)	Done
6b-i	pymavlink connection to ArduPilot MAVLink output	Done
6b-ii	Disable GPS; strip position from JSON bridge (IMU+baro only)	Done
6b-iii	AnyLoc → ArduPilot EKF3 via VISION_POSITION_ESTIMATE	Done
6b-iv	Flight commands via SET_POSITION_TARGET (replaces keyboard)	TODO
6c	HIGHRES_IMU from ArduPilot → localisation pipeline	TODO
6d	IMU fusion: AnyLoc anchor validator + VO quality gate	TODO
7	Full pipeline integrated in simulation	TODO
8	Deploy to real hardware	TODO

Key Design Decisions

ArduPilot before custom IMU fusion. On real hardware, IMU data arrives as MAVLink HIGHRES_IMU messages. Building against the MAVLink interface now means *zero code changes* at hardware deployment. ArduPilot’s own sensor models (bias drift, noise, temperature) are more realistic than analytical position derivatives.

No numpy — torch tensors throughout. The `isaac_sim_test` conda environment has a dual numpy conflict (pip numpy 2.x vs. conda numpy 1.26.4 from faiss-cpu). All VLAD pipeline operations use torch tensors; numpy is only touched at C-level FAISS call sites where the ABI is compatible.

Geo-constrained AnyLoc search. Unconstrained VLAD retrieval can jump to visually similar but geographically distant tiles (e.g. two similar road intersections 800 m apart). The 200 m window reduces the candidate set from 2,821 to ≈ 50 entries, making wrong-tile matches geometrically impossible.

Vision position, not bridge position. The JSON bridge’s “position” field is a GPS substitute in ArduPilot’s EKF3. Sending it would give the EKF ground truth, not AnyLoc estimates. It is intentionally absent; AnyLoc position arrives via `VISION_POSITION_ESTIMATE` instead — the same mechanism as Intel RealSense T265 or OptiTrack on real hardware.

Two MAVLink TCP ports. Each TCP port accepts exactly one client. The monitor (`run_mavlink.py`) uses port 5762; the vision bridge (`run_vision.py`) uses port 5763 (opened with `-out tcp:localhost:5763`).

Competition Readiness

The competition (2nd UAV Defense Challenge) requires GPS-denied autonomous UAV operation with multi-target vehicle detection and geo-location.

Table 7: Competition requirement gap analysis

Requirement	Current state	Gap
GPS-denied navigation	AnyLoc + VO (done)	Integrate ArduPilot autonomous flight
Civilian vehicle detection	yolov8l_visdrone.pt (done)	Verify car-roof logo detection
Military vehicle detection	Not in VisDrone classes	Collect aerial training data; fine-tune
Oil drums detection	Not in current model	Collect data; fine-tune
Amphibious APC detection	Not in current model	Collect data; fine-tune
Geo-location accuracy (≤ 50 m)	15–20 m anchor error	Meets requirement ($\varepsilon_i = 10$, full score)
GCS live display	matplotlib windows	Verify HDMI output to judges
Return-to-home	ArduPilot RTL (stub)	Wire up after Milestone 6b-iv

The localisation accuracy of ≈ 15 –20 m satisfies the competition’s tightest scoring bracket (≤ 50 m $\rightarrow$ full 10 position points per target).

Acknowledgements

- **AnyLoc** — “AnyLoc: Towards Universal Visual Place Recognition”, IRAL 2024. <https://github.com/AnyLoc/AnyLoc>
- **YOLOv8** — Ultralytics. <https://github.com/ultralytics/ultralytics>
- **ArduPilot** — Open-source autopilot. <https://ardupilot.org>
- **Isaac Sim 6.0** — NVIDIA. <https://developer.nvidia.com/isaac-sim>

- **Taiwan NLSC PHOTO2** — National Land Surveying and Mapping Center, National Land Surveying and Mapping Center, Taiwan.